Chat Models & Prompt Templates

LangChain v1.2.x (March 2026)

Comprehensive Guide

March 20, 2026

What We'll Cover

LangChain Pipeline Overview

Format Execute

Key Insight

Prompt Templates prepare input → Chat Models execute it → You get output. This section covers both components in depth.

Message Types: The Foundation

Core Message Types in LangChain

Every chat interaction uses typed messages. This prevents mistakes and enables structured handling.

- **HumanMessage**: User input
- **AIMessage**: Model responses
- **SystemMessage**: Instructions/context for the model
- **ToolMessage**: Results from tool calls

[Docs: langchain.schema]

HumanMessage & AIMessage

Basic Messages

```python
from langchain.schema import HumanMessage, AIMessage

# User speaks
msg1 = HumanMessage(content="What's 2+2?")

# Model responds
msg2 = AIMessage(content="The answer is 4.")

# Use in conversation
messages = [msg1, msg2]
```

Convention

Always wrap user input in `HumanMessage`. This makes intent explicit.

SystemMessage & ToolMessage

Specialized Messages

```python
from langchain.schema import SystemMessage, ToolMessage

# System: Tells model how to behave
sys = SystemMessage(
    content="You are a helpful math tutor."
)

# Tool: Response from external call
tool_result = ToolMessage(
    content="Result from calculation tool",
    tool_call_id="call_123"
)

messages = [sys, HumanMessage(...), tool_result]
```

Message Order

SystemMessage should come **first** in most cases. Models expect this order.

ChatModel Methods

Core Interface

All LangChain chat models follow this pattern: `invoke` — `batch` — `stream` — async versions

- **invoke()**: Single request, wait for response
- **batch()**: Multiple requests at once
- **stream()**: Real-time token streaming
- **ainvoke()**: Async single request
- **abatch()**: Async batch requests
- **astream()**: Async token streaming

[Docs: langchain.chat_models.base]

invoke() - Single Call

Basic invoke() Pattern

```python
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

model = ChatOpenAI(model="gpt-4")

# Single request
message = HumanMessage(content="Hello!")
response = model.invoke([message])

print(response.content)  # "Hello! How can I help?"
```

Note

`invoke()` blocks until the model responds. Use for simple scripts.

batch() - Multiple Calls

Batch Processing

```python
# Process multiple messages at once
messages_batch = [
    [HumanMessage(content="What is AI?")],
    [HumanMessage(content="What is ML?")],
    [HumanMessage(content="What is DL?")]
]

responses = model.batch(messages_batch)

for resp in responses:
    print(resp.content)
```

Efficiency

`batch()` is faster than sequential `invoke()` calls.

ChatOpenAI Setup

OpenAI Configuration

```python
from langchain_openai import ChatOpenAI

# Basic setup (uses OPENAI_API_KEY env var)
model = ChatOpenAI(
    model="gpt-4",
    temperature=0.7,
    api_key="sk-..."  # or use env var
)

# For gpt-3.5-turbo
model = ChatOpenAI(model="gpt-3.5-turbo")
```

[Docs: langchain_openai.ChatOpenAI]

ChatAnthropic Setup

Anthropic Claude Configuration

```python
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(
    model="claude-3-sonnet-20250219",
    temperature=0.7,
    api_key="sk-ant-..."  # or ANTHROPIC_API_KEY
)

# For Claude 3.5 Sonnet
model = ChatAnthropic(
    model="claude-3-5-sonnet-20241022"
)
```

[Docs: langchain_anthropic.ChatAnthropic]

ChatGoogleGenerativeAI Setup

Google Gemini Configuration

```python
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    temperature=0.7,
    api_key="AIzaSy..."  # or GOOGLE_API_KEY
)

# Alternative: gemini-1.5-pro
model = ChatGoogleGenerativeAI(
    model="gemini-1.5-pro"
)
```

[Docs: langchain_google_genai.ChatGoogleGenerativeAI]

ChatOllama Setup (Local)

Ollama Local Model

```python
from langchain_ollama import ChatOllama

# Requires: ollama pull llama2
model = ChatOllama(
    model="llama2",
    base_url="http://localhost:11434"
)

# For Mistral
model = ChatOllama(model="mistral")

# Streaming enabled by default
response = model.invoke([HumanMessage(...)])
```

Local Development

Perfect for testing without API costs. Install Ollama from ollama.ai

Key Model Parameters

Parameters Shape Behavior

These parameters control output quality, diversity, and constraints.

temperature	Randomness (0.0-2.0). Low = focused, High = creative
max_tokens	Output length limit. Plan for input + output
top_p	Nucleus sampling (0.0-1.0). Filters unlikely tokens
model	Which model to use (required)
stop	List of strings that end generation early

[Docs: Chat Model Parameters]

Temperature & max_tokens

Common Parameters

```
from langchain_openai import ChatOpenAI

# Focused, deterministic (for tasks)
model_task = ChatOpenAI(
    model="gpt-4",
    temperature=0.0,  # No randomness
    max_tokens=200    # Keep output short
)

# Creative (for brainstorming)
model_creative = ChatOpenAI(
    model="gpt-4",
    temperature=1.2,  # More diverse
    max_tokens=1000
)
```

Token Budget

max_tokens is output-only. Check: input + output ¡ model limit.

Stop Sequences & top_p

Advanced Parameters

```
# Stop generation at markers
model = ChatOpenAI(
    model="gpt-4",
    stop=["---", "STOP"],  # Stop if these appear
    top_p=0.9              # Only top 90% probability tokens
)

# Example: Stop at newline (useful for chat turns)
model = ChatOpenAI(
    model="gpt-4",
    stop=["\n"],
    temperature=0.7
)
```

Production Ready

Use stop sequences to enforce format (e.g., single response per call).

Why Prompt Templates?

Why Templates Matter

Templates prevent injection, enable composition, and make prompts testable.

- **Safety**: Variable escaping built-in
- **Reuse**: Same template, different inputs
- **Composition**: Combine templates
- **Clarity**: Intent is explicit

Question: Why not just use f-strings?
Answer: No validation, no composition, easier to mess up.

[Docs: langchain.prompts.PromptTemplate]

from_template() Method

Basic Template

```python
from langchain.prompts import PromptTemplate

# Define a reusable template
template = PromptTemplate.from_template(
    "Translate this to {language}: {text}"
)

# Use it
prompt = template.format(language="French", text="Hello")
print(prompt)
# Output: "Translate this to French: Hello"

# Or with invoke (preferred in chains)
prompt_str = template.format_prompt(
    language="Spanish",
    text="Good morning"
).to_string()
```

[Docs: PromptTemplate.from_template]

Input Variables & Partial Variables

Template Variables

```python
from langchain.prompts import PromptTemplate

# Explicit input variables
template = PromptTemplate(
    template="Role: {role}. Task: {task}",
    input_variables=["role", "task"]
)

# Partial: Pre-fill some variables
partial_template = template.partial(role="teacher")
prompt = partial_template.format(task="explain ML")
# Output: "Role: teacher. Task: explain ML"

# Useful for building complex templates
```

Pattern

Use `partial()` to build specialized templates from general ones.

ChatPromptTemplate vs PromptTemplate

Different Tools for Different Tasks

`PromptTemplate` works with strings. `ChatPromptTemplate` works with messages.

- **PromptTemplate**: Outputs a string (for generic text models)
- **ChatPromptTemplate**: Outputs a list of messages (for chat models)

Most modern work uses **ChatPromptTemplate**.

[Docs: langchain.prompts.ChatPromptTemplate]

from_messages() Method

Chat Template Setup

```python
from langchain.prompts import ChatPromptTemplate
from langchain.schema import HumanMessage, SystemMessage

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("user", "What is {topic}?")
])

# Format returns messages
messages = template.format_messages(topic="machine learning")
# [SystemMessage(...), HumanMessage(...)]

# Use with model
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4")
response = model.invoke(messages)
```

[Docs: ChatPromptTemplate.from_messages]

MessagesPlaceholder for History

Multi-turn Conversations

```
from langchain.prompts import (
    ChatPromptTemplate, MessagesPlaceholder)

template = ChatPromptTemplate.from_messages([
    ("system", "You are helpful."),
    MessagesPlaceholder(variable_name="history"),
    ("user", "{user_input}")
])

history = [
    HumanMessage(content="What is AI?"),
    AIMessage(content="AI is...")]

messages = template.format_messages(
    history=history,
    user_input="Tell me more.")
```

Key Pattern

`MessagesPlaceholder` inserts existing messages.

Few-Shot with Examples

Examples Improve Performance

```
from langchain.prompts import (
    FewShotPromptTemplate)

examples = [{"word": "happy", "antonym": "sad"}]

template = PromptTemplate(
    template="Word: {word}. Antonym: {antonym}",
    input_variables=["word", "antonym"])

prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=template)
```

[Docs: FewShotPromptTemplate]

Few-Shot Chat Messages

Few-Shot for Chat Models

```
from langchain.prompts import (
    FewShotChatMessagePromptTemplate,
    ChatPromptTemplate)

examples = [
    {"input": "happy", "output": "sad"}]

example_prompt = (
    ChatPromptTemplate.from_messages([
        ("user", "Word: {input}"),
        ("assistant", "Antonym: {output}")]))

few_shot = FewShotChatMessagePromptTemplate(
    examples=examples,
    example_prompt=example_prompt)
```

Few-Shot Chat: Final Prompt

Composing with System

```
final = ChatPromptTemplate.from_messages([
    ("system", "Find antonyms."),
    few_shot,
    ("user", "Word: {word}")
])
```

When to Use

Few-shot improves task-specific behavior. 3-5 examples often sufficient.

System Prompts

Setting Model Behavior

```python
from langchain.prompts import ChatPromptTemplate

# System prompt shapes behavior
template = ChatPromptTemplate.from_messages([
    ("system", "You are a Python expert. Answer with code."),
    ("user", "{question}")
])

messages = template.format_messages(
    question="How to read a file?"
)

model = ChatOpenAI(model="gpt-4")
response = model.invoke(messages)
```

System Message Placement

Always include SystemMessage first. Skipping it changes model behavior.

Persona Patterns

Detailed Personas

```
system_prompt = """You are a code reviewer.
Check for: bugs, security, performance.
Provide: issue, severity, fix."""

template = (
    ChatPromptTemplate.from_messages([
        ("system", system_prompt),
        ("user", "Review: {code}")
    ]))
```

Persona Clarity

Detailed personas reduce hallucination and improve consistency.

Composing Templates

Build Complex Prompts from Simple Ones

```python
from langchain.prompts import ChatPromptTemplate

system = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}.")
])

user_template = ChatPromptTemplate.from_messages([
    ("user", "{query}")
])

# Compose: system + user
combined = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}."),
    ("user", "{query}")
])

# Or dynamically
messages = system.messages + user_template.messages
```

[Docs: Composition patterns]

Reusable Template Library

Template Constants

```
CODE_REVIEW_TEMPLATE = (
    ChatPromptTemplate.from_messages([
        ("system", "Review code. Be thorough."),
        ("user", "{code}")
    ]))

SUMMARY_TEMPLATE = (
    ChatPromptTemplate.from_messages([
        ("system", "Summarize concisely."),
        ("user", "{text}")
    ]))
```

Using Templates in Chains

Composition Patterns

```python
# Use templates in chains
chain1 = CODE_REVIEW_TEMPLATE | model
chain2 = SUMMARY_TEMPLATE | model

# Parameterize for flexibility
def create_template(role, task):
    return ChatPromptTemplate.from_messages([
        ("system", f"You are {role}. {task}"),
        ("user", "{input}")
    ])
```

Modularity

Store in separate module for consistency.

stream() Method

Real-Time Token Streaming

```python
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

model = ChatOpenAI(model="gpt-4")

# Stream tokens as they arrive
for chunk in model.stream([HumanMessage(content="Hello")]):
    # chunk is an AIMessageChunk
    print(chunk.content, end="", flush=True)

print()  # newline at end
```

User Experience

Streaming makes apps feel responsive. Show tokens to user as they arrive.

Async Streaming

astream() for Async Code

```python
import asyncio
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

async def stream_response():
    model = ChatOpenAI(model="gpt-4")

    async for chunk in model.astream(
        [HumanMessage(content="Explain AI")]
    ):
        print(chunk.content, end="", flush=True)
    print()

# Run in async context
asyncio.run(stream_response())
```

[Docs: stream, astream]

ainvoke() - Async Single Request

Non-Blocking Calls

```python
import asyncio
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

async def main():
    model = ChatOpenAI(model="gpt-4")

    # Non-blocking await
    response = await model.ainvoke(
        [HumanMessage(content="What is AI?")]
    )

    print(response.content)

asyncio.run(main())
```

When to Use

Use async for web apps or concurrent processing.

abatch() - Parallel Processing

Concurrent Batch Processing

```python
import asyncio
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

async def main():
    model = ChatOpenAI(model="gpt-4")
    messages = [
        [HumanMessage(content="What is AI?")],
        [HumanMessage(content="What is ML?")],
        [HumanMessage(content="What is DL?")]]
    responses = await model.abatch(messages)
    for resp in responses:
        print(resp.content)

asyncio.run(main())
```

Parallelism

Much faster than sequential `ainvoke()`.

Common Errors

Error Types

Rate limits, auth failures, and timeouts are common. Always handle gracefully.

- **RateLimitError**: Too many requests
- **AuthenticationError**: Bad API key
- **APIConnectionError**: Network issue
- **APITimeoutError**: Request took too long
- **ValueError**: Bad input parameters

[Docs: Error Handling in LangChain]

Handling Rate Limits

Graceful Retry

```python
import time
from langchain_openai import ChatOpenAI
from openai import RateLimitError

def call_with_retry(messages, max_retries=3):
    for attempt in range(max_retries):
        try:
            return ChatOpenAI().invoke(messages)
        except RateLimitError:
            if attempt < max_retries - 1:
                wait = 2 ** attempt
                time.sleep(wait)
            else:
                raise
```

Best Practice

Exponential backoff: don't hammer API.

Handling Timeouts

Request Timeout

```python
from langchain_openai import ChatOpenAI
from openai import APITimeoutError

model = ChatOpenAI(
    model="gpt-4",
    request_timeout=30)

try:
    response = model.invoke([msg])
except APITimeoutError:
    print("Timeout. Retry or use shorter prompt.")
```

[Docs: request_timeout]

Top Mistakes

Avoid These Errors

Learning from common mistakes accelerates development.

1. Using plain strings instead of Message objects
2. Forgetting SystemMessage
3. Not handling token limits
4. Ignoring errors
5. Hardcoding API keys
6. Over-relying on temperature tuning

Mistake 1: Not Using Messages

Wrong Approach

```
# BAD: String instead of message
response = model.invoke("What is AI?")

# GOOD: Proper message
from langchain.schema import HumanMessage
response = model.invoke([HumanMessage(content="What is AI?")])
```

Why? Messages are typed, composable, and enable tool use.

Mistake 2: Forgetting System Prompt

Missing Context

```python
# BAD: No system message
messages = [HumanMessage(content="Explain quantum physics")]

# GOOD: Include system context
from langchain.schema import SystemMessage
messages = [
    SystemMessage(content="You explain physics simply."),
    HumanMessage(content="Explain quantum physics")
]
```

Impact: System prompts dramatically improve output quality.

Mistake 3: Ignoring Token Limits

Token Budget

```
# BAD: Could exceed limits
model = ChatOpenAI(
    model="gpt-3.5-turbo",
    max_tokens=3000  # Too close to 4k limit!
)

# GOOD: Leave buffer
model = ChatOpenAI(
    model="gpt-4",
    max_tokens=2000  # Safe margin
)

# Track: sum(input tokens) + max_tokens must be < model limit
```

[Docs: Model token limits]

Mistake 4: Hardcoding API Keys

Security Risk

```python
# BAD: Keys in code!
model = ChatOpenAI(api_key="sk-proj-abc123xyz...")

# GOOD: Use environment variables
import os
model = ChatOpenAI(
    model="gpt-4",
    api_key=os.getenv("OPENAI_API_KEY")
)

# Or rely on default (LangChain checks env automatically)
model = ChatOpenAI(model="gpt-4")
```

Rule

Never commit API keys. Use environment variables only.

Key Takeaways

Core Concepts

Remember these fundamentals for all LangChain work:

1. Messages (Human, AI, System, Tool) are the foundation
2. Chat models use invoke/batch/stream interface
3. Prompt templates are reusable and composable
4. System prompts shape behavior dramatically
5. Always handle errors gracefully
6. Use environment variables for secrets
7. Test with streaming before production

Next Steps

Section 02 builds on this: Chains, LCEL, and automation patterns.

Most-Used API Calls

Copy-Paste Ready

```python
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage
from langchain.prompts import ChatPromptTemplate

model = ChatOpenAI(model="gpt-4")

# Simple call
resp = model.invoke([HumanMessage(content="Hi")])

# Template-based
template = ChatPromptTemplate.from_messages([
    ("system", "You are helpful."),
    ("user", "{input}")
])
messages = template.format_messages(input="Hi")
resp = model.invoke(messages)

# Batch / Stream
responses = model.batch([msg1, msg2])
for chunk in model.stream([HumanMessage(...)]):
    print(chunk.content, end="")
```

Cheat Sheet: Parameters

Param	Use
temperature	0 = task, 1.0 = balanced, 1.5+ = creative
max_tokens	output length limit
top_p	nucleus sampling (0-1)
model	which LLM to use
stop	halt generation strings

Defaults

Safe by default. Adjust only when needed.

Questions to Test Understanding

1. What are the four message types in LangChain? When do you use each?
2. Why use PromptTemplate instead of f-strings?
3. What's the difference between invoke(), batch(), and stream()?
4. How do you initialize ChatOpenAI and ChatAnthropic?
5. What does `MessagesPlaceholder` do?

Self-Assessment

If you can't answer these, review relevant sections above.

More Questions

6. How do you build a multi-turn conversation with ChatPromptTemplate?
7. What's exponential backoff and why use it?
8. Why should you never hardcode API keys?
9. When should you use async (ainvoke, abatch, astream)?

Ready?

If all questions are clear, you're ready for Section 02: Chains & LCEL.